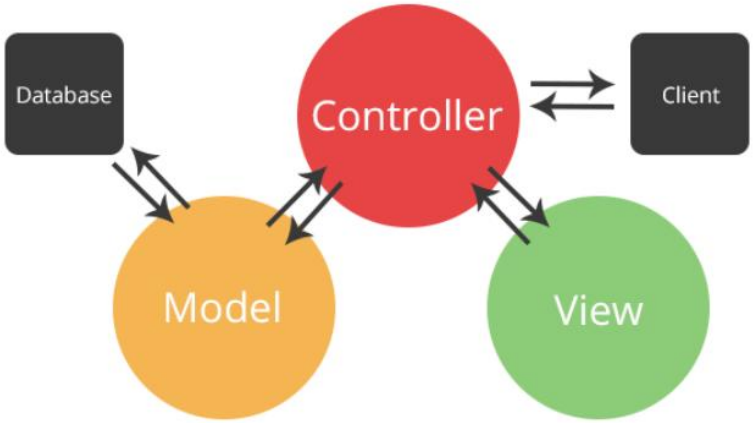
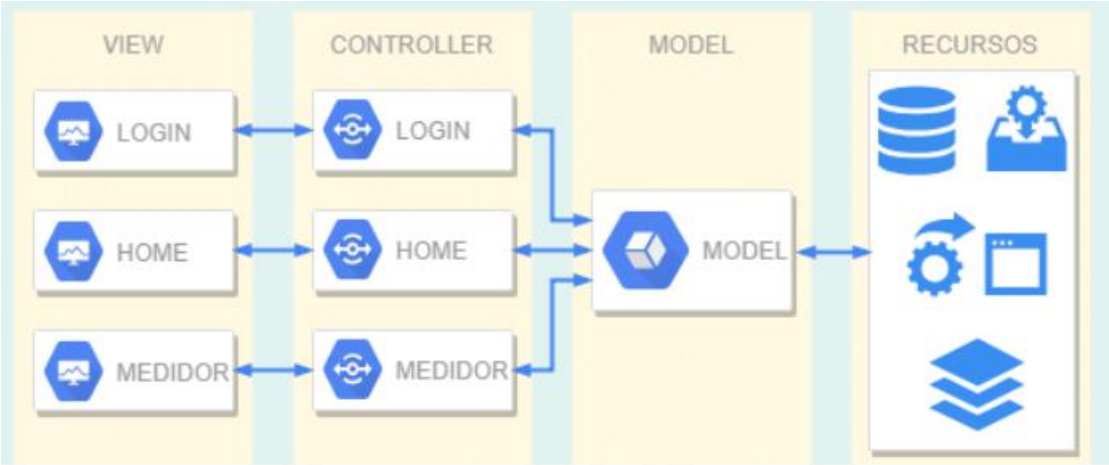
# MVC (Model - View - Controller)

A estrutura MVC (Model-View-Controller) surgiu no final da década de 1970, especificamente por volta de 1978-1979.

É um padrão de arquitetura de software que separa a aplicação em três camadas distintas para organizar o código, aumentar a coesão e facilitar a manutenção.

1. **Model (Modelo):** Gerencia dados e regras de negócio.
2. **View (Visão):** Interface gráfica (HTML, CSS, JS).
3. **Controller (Controlador):** Intermediário que recebe requisições e processa a lógica.



# Laravel

O Laravel é um **framework** de desenvolvimento web em PHP usado para criar sites e aplicações web de forma mais rápida, organizada e segura.

## Resumo simples

- É uma estrutura pronta para programar em PHP.
- Ajuda a organizar o código usando o padrão MVC (Model-View-Controller).
- Facilita tarefas comuns como login, rotas, banco de dados, autenticação e APIs.

## Principais características

- MVC: separa lógica, interface e dados.
- Eloquent ORM: facilita trabalhar com banco de dados.
- Blade: sistema de templates para criar páginas.
- Rotas simples: define URLs da aplicação facilmente.
- Segurança: proteção contra ataques comuns (CSRF, SQL Injection).

## Exemplo de uso

Com o Laravel é possível criar:
- Sistemas de login com autenticação e segurança
- APIs
- Lojas virtuais
- Sistemas administrativos
- Aplicações web completas
- SaaS
- Marketplace

## Ferramentas

- Laravel Cashier - Gerenciar assinaturas
- Stripe - pagamentos
- Laravel Sanctum - autenticação de API
- etc.

# Links

- https://laravel.com/
- https://laravel.com/docs/12.x
- https://getcomposer.org/
- https://getcomposer.org/download/

- https://bootswatch.com/
- https://getbootstrap.com/

## Extensões - VSCode:

- PHP Intelephense (PHP code intelligence for Visual Studio Code)
- Laravel Extension Pack (Winnie Lin)
- Laravel goto view (codingyu)
- Laravel Blade Snippets  (Winnie Lin)

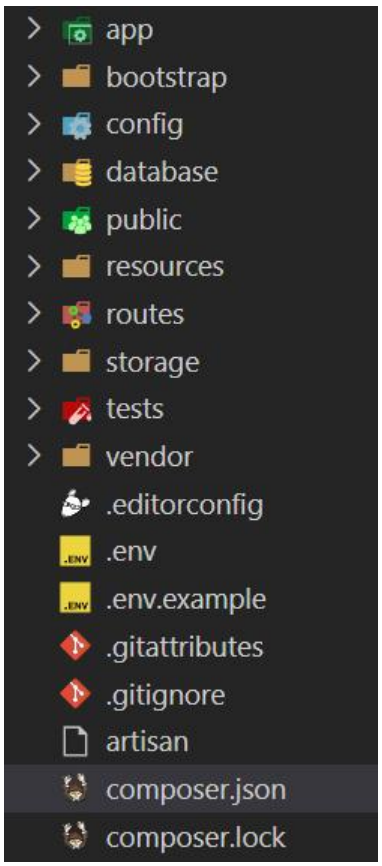
## Requisitos

- PHP 8.2 ou superior
- Composer

## Criando um projeto

- composer create-project "laravel/laravel:^10.0" nome_do_app
- composer create-project --prefer-dist laravel/laravel nome_do_app

## Estrutura de pastas



**Controller**
- app/Http/Controllers

**Models**
- app/Models

**Views**
- resources/views

**Rota**
- routes/web.php

# Artisan

- php artisan - é o acesso à CLI do laravel
- php artisan about - mostra um resumo completo das informações do seu projeto Laravel e do ambiente onde ele está rodando.
- php artisan config:show database - exibe informações sobre as funções disponíveis no laravel para conexão com base de dados

## Configuração:

Podemos configurar o projeto através dos arquivos localizados na pasta **config** ou através do arquivo **.env** disponível na raiz do projeto

- No arquivo: **config/app.php**
- Altere os códigos:

```
// 'env' => env('APP_ENV', 'production'),
'env' => env('APP_ENV', 'development'),

// 'debug' => (bool) env('APP_DEBUG', false),
'debug' => (bool) env('APP_DEBUG', true),
```

## Executando a aplicação:

- php artisan serve

## Rotas - Exemplos

```
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
   return view('welcome');
});

Route::get('/contato', function () {
   return view('teste.contatoTeste');
});

Route::get('/teste', function () {
   echo "Hello World!";
});
```

## Rota com name

```
Route::get('/', [Main::class, 'index'])->name('index');
```

## Vantagens de se utilizar rotas com nome (name)

Na linha:

```
Route::get('/', [Main::class, 'index'])->name('index');
```

- O método **->name('index')** define um nome para a rota.
- A principal vantagem é que você passa a se referir à rota pelo nome, e não pelo URL fixo.

### Vantagens de usar name() nas rotas

1- Facilita a geração de URLs

Em vez de fazer:

```
<a href="/">Home</a>
```

Você pode usar:

```
<a href="{{ route('index') }}">Home</a>
```

Ou no controller:

```
return redirect()->route('index');
```

Se um dia você mudar a URL de **/** para **/home**, por exemplo:

```
Route::get('/home', [Main::class, 'index'])->name('index');
```

Você não precisa alterar nada no restante do sistema, pois tudo continua chamando pelo nome da rota.

2- Melhor organização e manutenção

Em projetos maiores (como sistemas Laravel com muitos módulos), usar nomes evita:

- URLs “hardcoded” espalhadas pelo código
- Dificuldade na manutenção
- Erros ao alterar rotas

3- Essencial para redirecionamentos e formulários

Exemplo com redirecionamento:

```
return redirect()->route('index');
```

Exemplo em formulário Blade:

```
<form action="{{ route('index') }}" method="GET">
```

4- Ajuda na organização de grupos de rotas

- É muito comum usar prefixos de nome:

```
Route::prefix('admin')->name('admin.')->group(function () {
   Route::get('/dashboard', [AdminController::class, 'dashboard'])
                    ->name('dashboard');
});
```

- Nesse caso, o nome final da rota será:

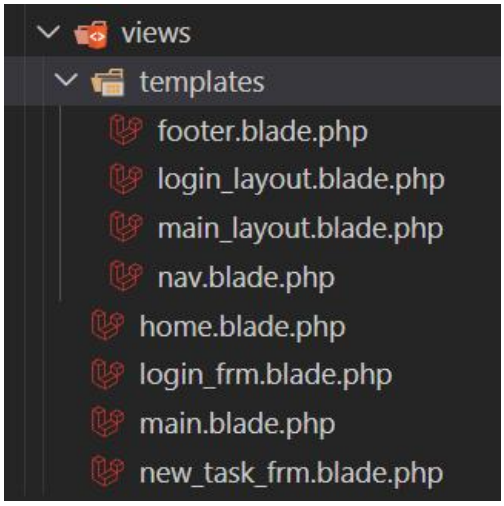
```
admin.dashboard
```

- Isso deixa o projeto muito mais estruturado.

**Resumindo Usar ->name():**
- Facilita manutenção
- Permite alterar URLs sem quebrar o sistema
- Torna o código mais organizado
- É essencial em projetos médios e grandes

# Template

No Laravel, o sistema de templates é gerenciado pelo motor Blade.
Ele funciona como uma camada de apresentação que permite criar layouts dinâmicos e reutilizáveis, facilitando a separação entre a lógica de programação (PHP) e o design da interface (HTML).




## Fragmentos de layout

### *- Nav [nav.blade.php]*

```
<nav class="navbar bg-body-tertiary">
    <div class="container-fluid">
        <a class="navbar-brand" href="#">Navbar</a>
    </div>
</nav>
```

### *- Footer [footer.blade.php]*

```
<footer>
        <h2>Footer</h2>
</footer>
```

### *- Template Raiz [main_layout.blade.php]*

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">

  <title>{{ $title ?? 'HOME' }}</title>

  {{-- asset() - vai entender que o caminho é relativo a pasta public --}}
  <link rel="stylesheet" href="{{ asset('assets/bootstrap.min.css') }}">
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-icons.css" rel="stylesheet">

</head>
<body>

   @include("templates.nav")

   @yield('content')

   @include("templates.footer")

   <script src="{{ asset('assets/bootstrap.bundle.min.js') }}"></script>
</body>
</html>
```

### *Herança visual (reutilizando views) - [home.blade.php]*

```
@extends('templates.main_layout')
@section('content')

   <h1>Home</h1>

@endsection
```

## Exercício:

- Reestruturar as views do projeto de extensão por meio do mecanismo de templates Blade do Laravel, visando a reutilização de layouts e componentes, além de melhorar a manutenção e padronização do código.

# Configurando base de dados

```
# Database configuration
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=HAE_FATEC_TESTE
DB_USERNAME=root
DB_PASSWORD=

SESSION_DRIVER=file

CACHE_STORE=file
```

## Testando a conexão com a base de dados

```
use Illuminate\Support\Facades\DB;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
   try {

      echo "Conexão bem-sucedida ao banco de dados!<br>" .
       DB::connection()->getDatabaseName();

   } catch (Exception $e) {

      return "Não foi possível conectar ao banco de dados. Erro: "
                  . $e->getMessage();
   }
});
```